DjangoSentinel User Manual

Enterprise AI Security Scanner for Python/Django — End-user guide

DjangoSentinel v1.0.0 — User Manual

Table of Contents

1. [Installation](#installation)
2. [Quick Start](#quick-start)
3. [CLI Commands](#cli-commands)
4. [Scan Modes](#scan-modes)
5. [Auto-Fix](#auto-fix)
6. [Git Hooks](#git-hooks)
7. [Output Formats](#output-formats)
8. [CI Integration](#ci-integration)

Installation

```
# Install from PyPI
pip install django-sentinel

# Or install in editable mode from source
git clone https://github.com/yourusername/django-sentinel.git
cd django-sentinel
pip install -e .
```

Requirements for AI features:

```
export ANTHROPIC_API_KEY="your-claude-key"
# AI fixes, explanations, and pattern scanning require ANTHROPIC_API_KEY
```

Quick Start

```
# Full scan of current directory
sentinel scan .

# Diff scan (changed lines only — ideal for PRs)
sentinel scan . --diff

# Scan and auto-fix safe vulnerabilities
sentinel scan . --fix

# Scan with AI fixes and explanations
sentinel scan . --fix --ai
```

```
# Scan only staged changes (pre-commit)
sentinel scan . --diff --staged

# Skip certain scanners
sentinel scan . --skip-ast --skip-deps

# Minimum confidence threshold
sentinel scan . --min-confidence 0.7

# Fail CI on critical findings
sentinel scan . --fail-on critical
```

CLI Commands

`sentinel scan`

Runs the security scanner.

Option	Description
`--format`	Output format: `terminal` (default), `json`, `html`, `sarif`
`-o` / `--output`	Save report to file
`--no-color`	Disable ANSI colors
`--diff`	Scan only changed lines (git diff)
`--staged`	With `--diff`: scan staged changes only
`--base BRANCH`	With `--diff`: compare against branch
`--fix`	Auto-fix safe vulnerabilities
`--fix --ai`	Rule-based fix + AI for complex issues
`--min-confidence FLOAT`	Minimum confidence threshold (0.0-1.0)
`--fail-on SEVERITY`	Exit 1 if finding at this severity or above
`--skip-ast` / `--skip-logic` / `--skip-deps` / `--skip-taint`	Skip specific scanners
`--ai`	Enable AI-powered fixes/explanations
`--ai-deep`	Run AI pattern scanner on every file
`--ai-explain`	Generate AI explanation for every finding

`sentinel fix`

Scan and apply all fixes.

Option	Description
`--dry-run`	Preview fixes without applying
`--no-backup`	Skip creating backups

`--ai`	Use AI for complex fixes

`sentinel explain`

Generate AI explanation for a specific code line.

```
sentinel explain views.py 42
```

Options: `--no-color`

`sentinel install-hooks`

Install git pre-commit and pre-push hooks.

```
sentinel install-hooks
# Or: sentinel install-hooks --hooks pre-commit
```

Options: `--hooks pre-commit | pre-push` (default: both)

`sentinel uninstall-hooks`

Remove installed git hooks.

`sentinel hook-status`

Check hook installation status.

`sentinel install-hooks --help`

Show help for any command.

Scan Modes

Full Scan (default)

Scans all Python and HTML files in the project using:

- **Django Scanner** — settings, views, models, URLs, templates, secrets
- **AST Scanner** — structural patterns (random, MD5, timing attacks, etc.)
- **Business Logic Scanner** — race conditions, privilege escalation, workflow bypass
- **Dependency Scanner** — CVE database for requirements.txt
- **Data-Flow Taint Analysis** — source→sink tracking with sanitizer detection

Diff Scan (`--diff`)

Scans ONLY lines changed in the git diff. Ideal for PR reviews.

```
sentinel scan . --diff
sentinel scan . --diff --base main
sentinel scan . --diff --staged
sentinel scan . --diff-file pr.patch
```

AI-Enhanced Scanning

```
sentinel scan . --ai              # AI explanations for findings
sentinel scan . --ai-deep         # AI pattern scanner on all files
sentinel scan . --fix --ai        # Auto-fix + AI for complex issues
```

Auto-Fix

Rule-based fixes are applied automatically for vulnerabilities where `manual_review_required=False`. AI-generated fixes require `ANTHROPIC_API_KEY`.

Fix priority:

1. Rule-based fixes (fast, deterministic) — always applied if available
2. AI-generated fixes (complex, multi-line) — requires ANTHROPIC_API_KEY
3. Manual review — flagged issues with `manual_review_required=True`

Backup: Backups are saved to `.sentinel_backups/YYYYMMDD_HHMMSS/` before any file modification.

Git Hooks

Pre-commit hook: Scans staged Python/HTML files. Blocks commit if CRITICAL vulnerabilities found.

Pre-push hook: Full project scan. Blocks push if CRITICAL or HIGH vulnerabilities found.

```
sentinel install-hooks    # Install both hooks
sentinel install-hooks --hooks pre-commit    # Install only pre-commit
```

Hooks can be uninstalled:

```
sentinel uninstall-hooks
```

Output Formats

Terminal (default)

Colored output with vulnerability summary, severity breakdown, and per-finding details.

JSON

Machine-readable output with full vulnerability data, confidence scores, and metadata.

```
sentinel scan . --format json -o report.json
```

HTML

Full HTML report with vulnerability grid, severity badges, fix blocks, and searchable interface.

```
sentinel scan . --format html -o report.html
```

SARIF

SARIF 2.1.0 format for GitHub Code Scanning upload.

```
sentinel scan . --format sarif -o report.sarif
```

Diff Report

Compact diff-aware view showing changed files, added lines, and findings scoped to the diff.

```
sentinel scan . --diff --format terminal
```

CI Integration

GitHub Actions

```
- name: DjangoSentinel Security Scan
  run: sentinel scan . --fail-on high
```

Exit Codes

- `0` — scan completed successfully
- `1` — `--fail-on` threshold exceeded (unfixed findings at or above severity)

SARIF Upload

```
- name: Upload SARIF
  uses: github/codeql-action/upload-sarif@v2
  with:
    sarif_file: report.sarif
```

---

9. Docker Containerized Deployment

DjangoSentinel can be run inside a Docker container for an isolated, reproducible environment. The same code base supports two environments: a small **production** image that runs the `sentinel` command, and a **development** image (editable install plus test tools) for working on DjangoSentinel itself.

Prerequisites

- [Docker](https://docs.docker.com/engine/install/) with Docker Compose v2

Quick Start

```
# 1. Create your environment file and add ONE AI key (optional - the scan works without AI)
cp .env.example .env

# 2. Scan the current folder (production environment)
docker compose run --rm sentinel scan .

# Scan another project and write a SARIF report
SCAN_PATH=/path/to/django/project docker compose run --rm sentinel scan . --format sarif -o report.sarif

# 3. Switch to the development environment
docker compose --profile dev run --rm dev            # runs the test-suite
docker compose --profile dev run --rm dev bash       # opens a shell
```

The project to scan is mounted at `/src` inside the container. Both services are one-shot jobs, so they do not restart on their own.

Without Docker Compose

```
docker build -t django-sentinel .
docker run --rm -v "$(pwd)":/src -e ANTHROPIC_API_KEY django-sentinel scan .
```

Environment Variables

Variable	Purpose
`ANTHROPIC_API_KEY`, `OPENAI_API_KEY` or `GOOGLE_API_KEY`	AI provider (only one is needed)
`GITHUB_TOKEN`, `GITHUB_REPOSITORY`, `GITHUB_SHA`, `GITHUB_REF`	GitHub pull-request integration
`SCAN_PATH`	Folder to scan (defaults to the current folder)